Informe post-incidente - Mantenimiento de VoltTech

Bases de Datos II · Comisión 4 · Joaquin Antonio Mendez Reynoso · PostgreSQL 17.10 · Base: volttech_u6 · 10 de noviembre de 2026

Repositorio: https://github.com/VIKROOM/db_Mendez_COM4_U6.git

1. Respaldo utilizado

Dato	Valor
Archivo	volttech_u6_20261110_203628.dump (formato custom -Fc, compresión -Z 6)
Fecha y hora	2026-11-10, ejecución 20:36:28–20:36:31 (2,9 s)
Tamaño e integridad	11.332.222 bytes (10,81 MB), 58 objetos; SHA-256 621D798175B2077F…64E92C13
Identidad	mant_backup (solo lectura + INSERT en bitácora), sin contraseña en el script

2. RPO y RTO: previsto vs. observado

Métrica	Previsto	Observado	Cumple
RPO (pérdida máxima de datos)	≤ 24 h (respaldo diario)	58 s (0 min 58 s) de antigüedad	Sí
RTO (tiempo de recuperación)	≤ 20 min	7,60 s (createdb + pg_restore + validaciones)	Sí

El RTO se midió con cronómetro desde el instante anterior a createdb hasta completar la última validación. El RPO se calculó como la diferencia entre el inicio del simulacro y la fecha del .dump.

3. Resultados de la validación

Restauración en volttech_u6_restore_test con mant_restore (CREATEDB, sin superusuario), pg_restore --no-owner --no-privileges --exit-on-error -j 4:

- Conteos coincidentes 7/7 contra el snapshot previo: 2 categorías, 20.001 clientes, 50.002 productos, 200.001 pedidos, 800.002 líneas, suma de cantidades 1.599.736 e importe 800.285.374.188,00.
- productos_invalidos = 0 y detalles_invalidos = 0 (precio, stock y cantidad dentro de rango).
- Huérfanos referenciales = 0 en pedido→cliente y línea→pedido/producto.
- 16 restricciones declarativas restauradas y vista de compatibilidad public.detalle_pedido presente.
- Objetos de la app VoltTech verificados: vistas v_productos_vigentes, v_pedidos_cliente y v_detalle_pedido_producto, vista materializada mv_facturacion_categoria_mes (repoblada con REFRESH), procedimientos registrar_pedido y ajustar_stock, y extensión pg_trgm.
- volttech_u6 no se tocó: el script aborta si el destino no termina en _restore_test.

4. Problema real encontrado

pg_dump restaura las vistas materializadas sin datos: mv_facturacion_categoria_mes quedaba como definición vacía en la base restaurada tras un pg_restore impecable (0 errores y 7/7 conteos). No es un fallo del respaldo sino un comportamiento estándar que el procedimiento debía cubrir: un operador que valide solo conteos de tablas entregaría reportes vacíos. Se agregó REFRESH MATERIALIZED VIEW a las validaciones de restauración y quedó registrado en la evidencia: 26 filas tras repoblar (restore_validaciones.txt, sección 8).

5. Mejora concreta

Para la siguiente ejecución: (1) programar backup_volttech.ps1 y health_check.sql en el Programador de tareas de Windows (02:00) para que el RPO de 24 h no dependa de la ejecución manual, y (2) convertir la sección 6 del chequeo de salud (antigüedad del último respaldo OK) en una alerta explícita cuando supere las 20 h, margen sobre el RPO; hoy solo informa la antigüedad.

6. Uso de IA (OpenCode) y verificación

OpenCode generó la especificación, los scripts de respaldo, monitoreo, mantenimiento y restauración, y este informe. Nada se aceptó sin ejecutarlo contra el motor real: se cargó volttech_u6 sobre la app VoltTech del equipo (800.002 líneas de pedido), se ejecutaron los cuatro artefactos, la prueba de falla de pg_dump (exit 1, sin .dump válido) y el simulacro completo. La intervención humana revisó 12 decisiones de la IA documentadas en BITACORA_IA.md: credenciales y pgpass, rechazo de VACUUM FULL, vista de compatibilidad detalle_pedido, límite de conexiones para pg_restore -j 4, repoblado de la vista materializada y encoding de las evidencias, entre otras. La autoevaluación soporte/verificar_entrega.ps1 terminó con 102 PASS / 0 FAIL, y las salidas crudas quedaron en evidencias/ (restore_metrics.json, logs, health checks y listado del dump).